

20 – Fitness Tracker Object-Oriented Programming Example

Un Ejemplo Extendido

El colapso de los precios de la vivienda en Estados Unidos ayudó a desencadenar una crisis económica internacional en el otoño de 2008. Uno de los factores contribuyentes fue que demasiados propietarios habían adquirido hipotecas que terminaron teniendo consecuencias inesperadas.

Al comienzo, las hipotecas eran bestias relativamente simples. Los compradores pedían dinero prestado a un banco y hacían un pago fijo cada mes durante la vida de la hipoteca, que típicamente oscilaba entre 15 y 30 años. Al final de ese período, el banco había recibido de vuelta el préstamo inicial (el principal) más los intereses, y el propietario era dueño de la casa libre y sin gravámenes.

Hacia finales del siglo XX, las hipotecas comenzaron a volverse mucho más complicadas. Las personas podían obtener tasas de interés más bajas pagando "puntos" al prestamista en el momento en que adquirían la hipoteca. Un punto es un pago en efectivo del 1% del valor del préstamo. Las personas podían adquirir hipotecas que eran "solo de interés" por un período de tiempo. Es decir, durante algunos meses al inicio del préstamo, el prestatario pagaba solo el interés acumulado y nada del principal. Otros préstamos involucraban múltiples tasas. Típicamente, la tasa inicial (llamada "tasa de enganche") era baja, y luego subía con el tiempo. Muchos de estos préstamos tenían tasas variables: la tasa a pagar después del período inicial variaba dependiendo de algún índice destinado a reflejar el costo para el prestamista de pedir prestado en el mercado crediticio mayorista.

En principio, dar a los consumidores una variedad de opciones es algo bueno. Sin embargo, los prestamistas inescrupulosos no siempre fueron cuidadosos en explicar completamente las posibles implicaciones a largo plazo de las varias opciones, y algunos prestatarios tomaron decisiones que resultaron tener consecuencias terribles.

Construyamos un programa que examine los costos de tres tipos de hipotecas:

- Una hipoteca de tasa fija sin puntos.

- Una hipoteca de tasa fija con puntos.
- Una hipoteca con una tasa de enganche inicial seguida de una tasa más alta durante el resto del período

El punto de este ejercicio es proporcionar algo de experiencia en el desarrollo incremental de un conjunto de clases relacionadas, no convertirte en un experto en hipotecas.

Estructuraremos nuestro código para incluir una clase `Mortgage` y subclases correspondientes a cada uno de los tres tipos de hipotecas listados arriba. El siguiente código contiene la **clase abstracta** `Mortgage`. Esta clase contiene métodos que son compartidos por cada una de sus subclases, pero no está destinada a ser instanciada directamente. Es decir, no se crearán objetos de tipo `Mortgage`.

La función `find_payment` en la parte superior de la figura calcula el tamaño del pago mensual fijo necesario para saldar el préstamo, incluyendo intereses, al final de su plazo. Hace esto usando una expresión de forma cerrada bien conocida. Esta expresión no es difícil de derivar, pero es mucho más fácil simplemente buscarla y es más probable que sea correcta que una derivada sobre la marcha.

Ten en mente, sin embargo, que no todo lo que descubres en la web (o incluso en libros de texto) es correcto. Cuando tu código incorpora una fórmula que has buscado, asegúrate de que:

- Has tomado la fórmula de una fuente confiable. Consultamos múltiples fuentes confiables, todas las cuales contenían fórmulas equivalentes.
- Entiendes completamente el significado de todas las variables en la fórmula.
- Pruebas tu implementación contra ejemplos tomados de fuentes confiables. Después de implementar esta función, la probamos comparando nuestros resultados con los resultados proporcionados por una calculadora disponible en la web.

Clase base Mortgage

```
1 def find_payment(loan, r, m):
2     """Asume: loan y r son floats, m un int
3         Devuelve el pago mensual para una hipoteca de tamaño
4         loan a una tasa mensual de r por m meses"""
5     return loan * ((r * (1+r) ** m) / ((1+r) ** m - 1))
6
7 class Mortgage(object):
8     """Clase abstracta para construir diferentes tipos de hipotecas"""
9     def __init__(self, loan, ann_rate, months):
```

```

10         """Asume: loan y ann_rate son floats, months un int
11         Crea una nueva hipoteca de tamaño loan, duración months, y
12         tasa anual ann_rate"""
13         self._loan = loan
14         self._rate = ann_rate / 12
15         self._months = months
16         self._paid = [0.0]
17         self._outstanding = [loan]
18         self._payment = find_payment(loan, self._rate, months)
19         self._legend = None #descripción de la hipoteca
20
21     def make_payment(self):
22         """Hacer un pago"""
23         self._paid.append(self._payment)
24         reduction = self._payment - self._outstanding[-1] * self._rate
25         self._outstanding.append(self._outstanding[-1] - reduction)
26
27     def get_total_paid(self):
28         """Devolver la cantidad total pagada hasta ahora"""
29         return sum(self._paid)
30
31     def __str__(self):
32         return self._legend

```

Mirando `__init__`, vemos que todas las instancias de `Mortgage` tendrán variables de instancia correspondientes a la cantidad inicial del préstamo, la tasa de interés mensual, la duración del préstamo en meses, una lista de pagos que se han hecho al comienzo de cada mes (la lista comienza con 0, ya que no se han hecho pagos al inicio del primer mes), una lista con el saldo del préstamo que está pendiente al comienzo de cada mes, la cantidad de dinero que se debe pagar cada mes (inicializada usando el valor devuelto por la función `find_payment`) y una descripción de la hipoteca (que inicialmente tiene un valor de `None`). La operación `__init__` de cada subclase de `Mortgage` se espera que comience llamando a `Mortgage.__init__`, y luego inicialice `self._legend` con una descripción apropiada de esa subclase.

El método `make_payment` se usa para registrar los pagos de la hipoteca. Parte de cada pago cubre la cantidad de interés que se debe sobre el saldo pendiente del préstamo, y el resto del pago se usa para reducir el saldo del préstamo. Por eso `make_payment` actualiza tanto `self._paid` como `self._outstanding`.

El método `get_total_paid` usa la función incorporada de Python `sum`, que devuelve la suma de una secuencia de números. Si la secuencia contiene un `non-number`, se genera una excepción.

El siguiente código contiene clases que implementan tres tipos de hipotecas. Las clases `Fixed` y `Fixed_with_pts` anulan `__init__` e heredan

los otros tres métodos de `Mortgage`. La clase `Two_rate` trata una hipoteca como la concatenación de dos préstamos, cada uno a una tasa de interés diferente. (Dado que `self._paid` se inicializa como una lista con un elemento, contiene un elemento más que el número de pagos que se han hecho. Es por eso que el método `make_payment` compara `len(self._paid)` con `self._teaser_months + 1`.)

También contiene una función que calcula e imprime el costo total de cada tipo de hipoteca para un conjunto de muestra de parámetros. Comienza creando una hipoteca de cada tipo. Luego hace un pago mensual en cada una durante un número dado de años. Finalmente, imprime la cantidad total de los pagos hechos para cada préstamo.

Ahora estamos, finalmente, listos para comparar diferentes hipotecas:

```
1 compare_mortgages(amt=200000, years=30, fixed_rate=0.035,  
2                     pts = 2, pts_rate=0.03, var_rate1=0.03,  
3                     var_rate2=0.05, var_months=60)
```

Nota que usamos argumentos de palabra clave en lugar de argumentos posicionales en la invocación de `compare_mortgages`. Hicimos esto porque `compare_mortgages` tiene un gran número de parámetros formales del mismo tipo, y usar argumentos de palabra clave hace más fácil asegurar que estamos suministrando los valores reales pretendidos a cada uno de los parámetros formales.

Cuando el código se ejecuta, imprime:

```
1 Fixed, 3.5%  
2 Total payments = $323,312  
3  
4 Fixed, 3.0%, 2 points  
5 Total payments = $307,555  
6  
7 3.0% for 60 months, then 5.0%  
8 Total payments = $362,435
```

Subclases de Mortgage

```
1 class Fixed(Mortgage):  
2     def __init__(self, loan, r, months):  
3         Mortgage.__init__(self, loan, r, months)  
4         self._legend = f'Fixed, {r*100:.1f}%'  
5  
6 class Fixed_with_pts(Mortgage):  
7     def __init__(self, loan, r, months, pts):  
8         Mortgage.__init__(self, loan, r, months)  
9         self._pts = pts
```

```

10         self._paid = [loan*(pts/100)]
11         self._legend = f'Fixed, {r*100:.1f}%, {pts} points'
12
13     class Two_rate(Mortgage):
14         def __init__(self, loan, r, months, teaser_rate, teaser_months):
15             Mortgage.__init__(self, loan, teaser_rate, months)
16             self._teaser_months = teaser_months
17             self._teaser_rate = teaser_rate
18             self._nextRate = r/12
19             self._legend = (f'{100*teaser_rate:.1f}% for ' +
20                             f'{self._teaser_months} months, then {100*r:.1f}%')
21
22         def make_payment(self):
23             if len(self._paid) == self._teaser_months + 1:
24                 self._rate = self._nextRate
25                 self._payment = find_payment(self._outstanding[-1],
26                                             self._rate,
27                                             self._months - self._teaser_months)
28                 Mortgage.make_payment(self)
29
30     def compare_mortgages(amt, years, fixed_rate, pts, pts_rate,
31                           var_rate1, var_rate2, var_months):
32
33         tot_months = years * 12
34
35         fixed1 = Fixed(amt, fixed_rate, tot_months)
36
37         fixed2 = Fixed_with_pts(amt, pts_rate, tot_months, pts)
38
39         two_rate = Two_rate(amt, var_rate2, tot_months, var_rate1, var_months)
40
41         morts = [fixed1, fixed2, two_rate]
42
43         for m in range(tot_months):
44             for mort in morts:
45                 mort.make_payment()
46
47         for m in morts:
48             print(m)
49             print(f'Total payments = ${m.get_total_paid():,.0f}')

```

A primera vista, los resultados parecen bastante concluyentes. El préstamo de tasa variable es una mala idea (para el prestatario, no para el prestamista) y el préstamo de tasa fija con puntos cuesta menos. Es importante notar, sin embargo, que el costo total no es la única métrica por la cual las hipotecas deben ser juzgadas. Por ejemplo, un prestatario que espera tener un mayor ingreso en el futuro puede estar dispuesto a pagar más en los últimos años para reducir la carga de los pagos al principio.

Esto sugiere que en lugar de mirar un solo número, deberíamos mirar los pagos a lo largo del tiempo. Esto a su vez sugiere que nuestro

programa debería estar produciendo gráficos diseñados para mostrar cómo se comporta la hipoteca a lo largo del tiempo.

Workout Tracker Example

Supongamos que queremos escribir un programa para seguir nuestros entrenamientos.

Debemos identificar los distintos tipo de entrenamientos, por ej: correr, nadar, bicicleta, etc.

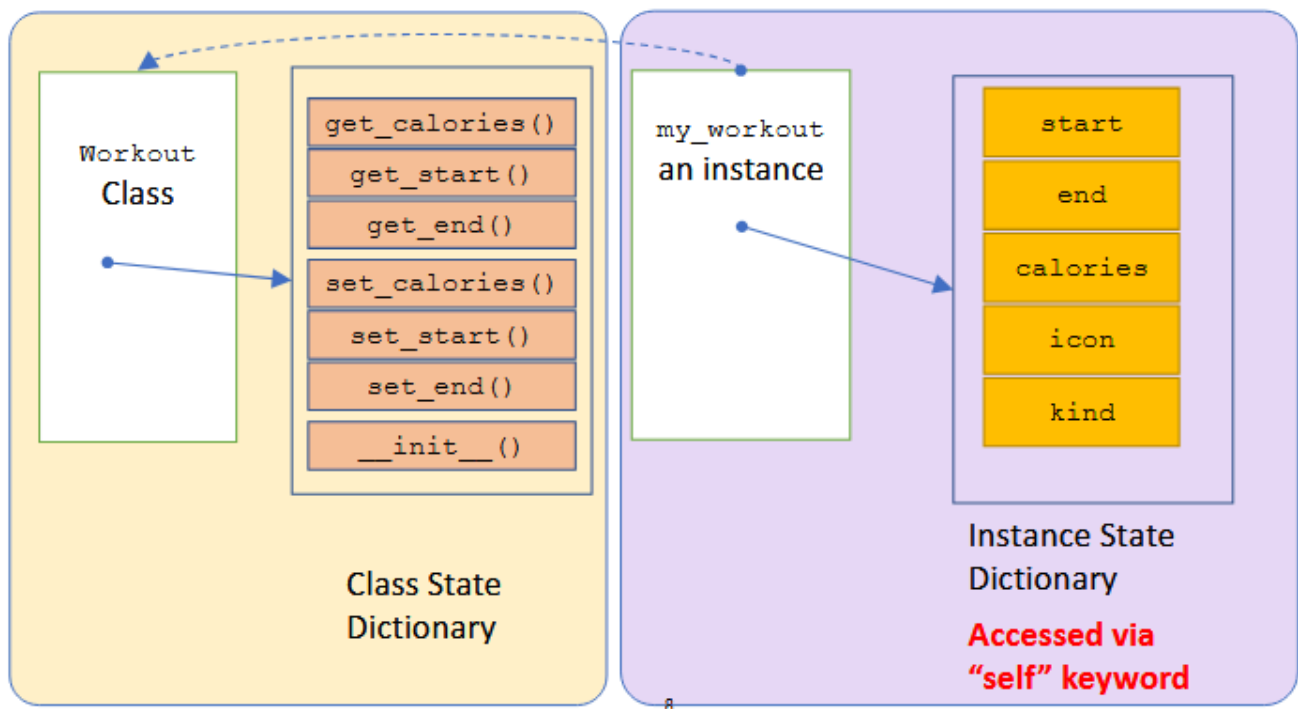
Ahora debemos identificar sus propiedades, como las propiedades globales, las específicas para correr, para el nado, etc.

Esas propiedades pueden ser atributos de datos o atributos funcionales. Entonces ahora definamos la clase `Workout`.

```
1 class Workout(object):
2     def __init__(self, start, end, calories):
3         self.start = start
4         self.end = end
5         self.calories = calories
6         self.icon = '😓'
7         self.kind = 'Workout'
8
9 my_workout = Workout("9/30/2021 1:35 PM", "9/30/2021 1:57 PM", 200)
```

Definamos unos getters y setters para la clase:

```
1     def get_calories(self):
2         return self.calories
3
4     def get_start(self):
5         return self.start
6
7     def get_end(self):
8         return self.end
9
10    def set_calories(self, calories):
11        self.calories = calories
12
13    def set_start(self, start):
14        self.start = start
15
16    def set_end(self, end):
17        self.end = end
```

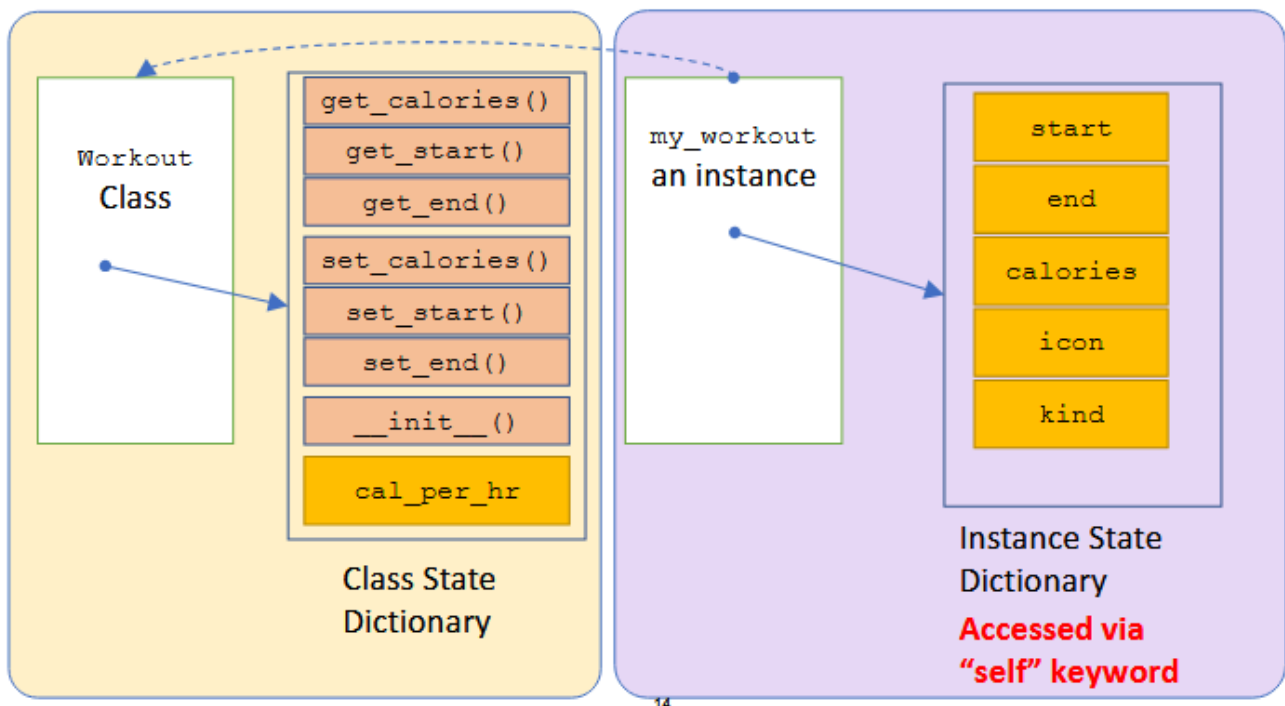


Es mejor que usemos los getters de la clase para acceder a los datos ya que favorece a la ocultacion de informacion.

```
1  # ❌ Mal
2  my_workout.calories
3  # ✅ Bien
4  my_workout.get_calories()
```

Ahora cambiemos la implementacion de nuestra clase:

```
1  from dateutil import parser # Python Library para manejar datetime
2
3  class Workout:
4      cal_per_hr = 200 # variable de clase, todas las instancias pueden
      leerlo
5
6      def __init__(self, start, end, calories = None):
7          self.start = parser.parse(start) # Son objetos del tipo datetime
8          self.end = parser.parse(end) # Son objetos del tipo datetime
9          self.calories = calories
10         self.icon = '😓😓'
11         self.kind = 'Workout'
12
13         def get_calories(self): # Si las calorias no fueron especificadas se
            estiman basadas en el tiempo transcurrido
14             if self.calories is None:
15                 dur = (self.end - self.start).total_seconds() / 3600
16                 return Workout.cal_per_hr * dur
17             return self.calories
```



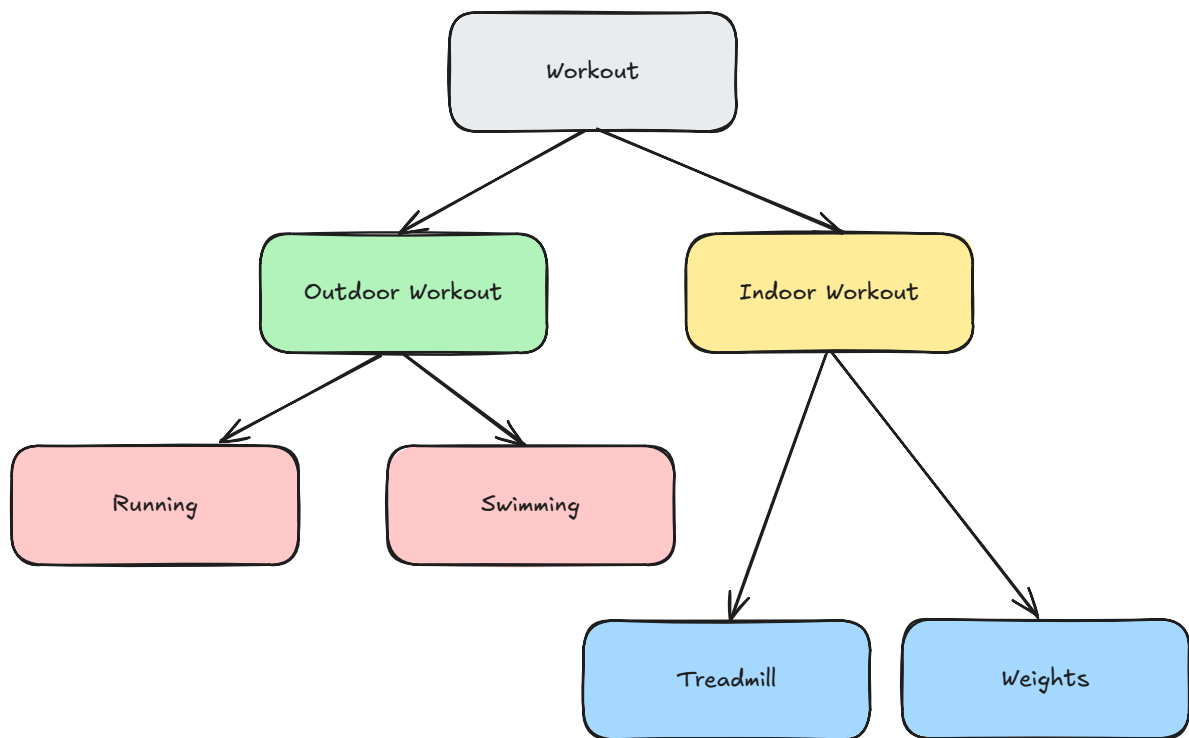
⚠ Guarda con las variables de instancia

Si llegas a cambiar `cal_per_hr` acórdete que se va a cambiar en todas las instancias de `Workout`.

Jerarquías de Clases: el arte de la especialización

En programación orientada a objetos, el concepto de jerarquías de clases permite organizar el código como si se tratara de un árbol genealógico: clases generales (padre) dan origen a clases más específicas (hijas), que heredan comportamientos comunes y suman sus propias particularidades.

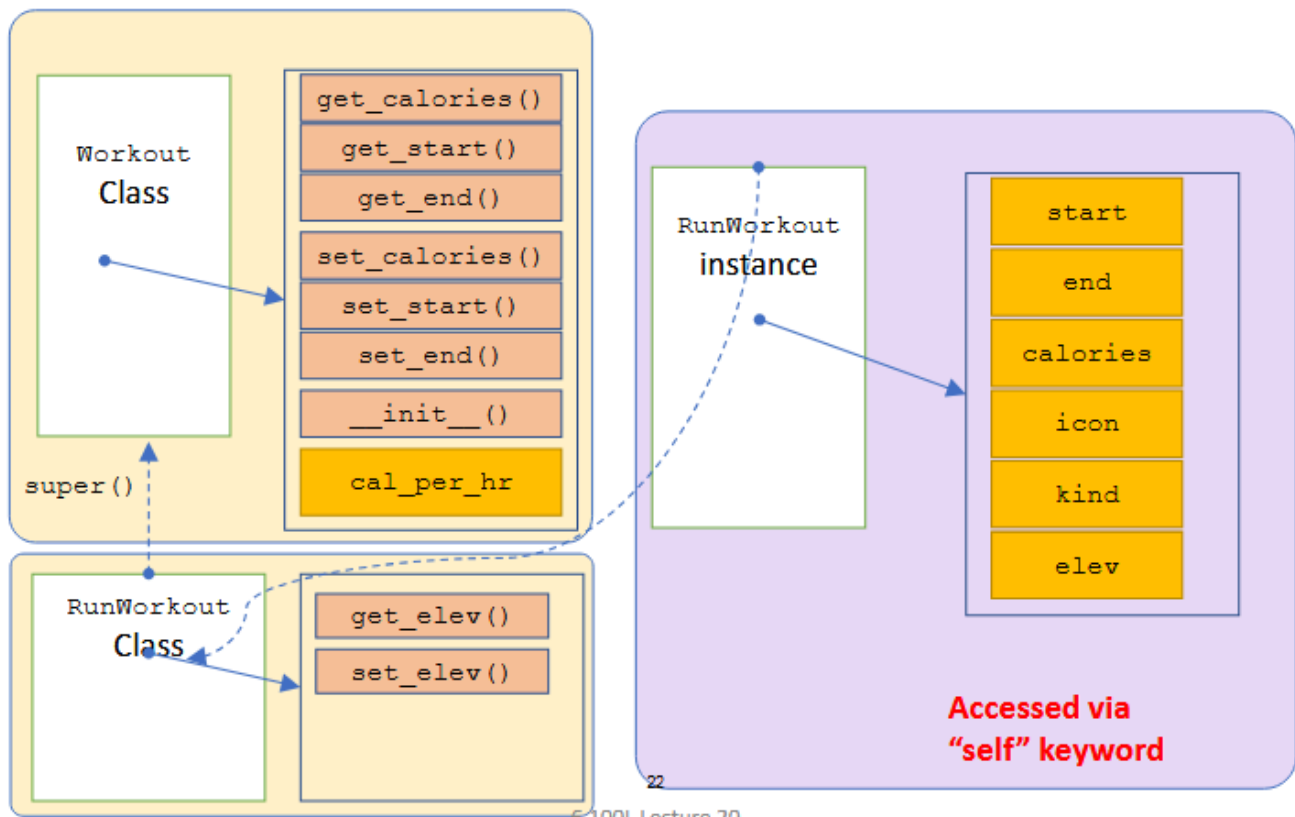
En el caso del `Fitness Tracker`, `Workout` es la clase madre. Representa cualquier tipo de entrenamiento, con atributos como hora de inicio, fin, y calorías quemadas. Sin embargo, actividades como correr o nadar contienen información específica (como elevación o tipo de brazada). Para modelar esta diversidad, creamos subclases.



Definiendo una Subclase: RunWorkout

```
1 class RunWorkout(Workout):
2     def __init__(self, start, end, elev=0, calories=None):
3         super().__init__(start, end, calories)
4         self.icon = '🏃'
5         self.kind = 'Running'
6         self.elev = elev
7
8     def get_elev(self):
9         return self.elev
10
11    def set_elev(self, e):
12        self.elev = e
```

Aquí, `RunWorkout` hereda todos los métodos y atributos de `Workout`, pero le suma un dato clave para actividades al aire libre: la **elevación**. Este ejemplo demuestra cómo reutilizar código con `super()` y extenderlo con nuevos comportamientos.



6-1001 Lecture 20

¿Dónde puede usarse una subclase?

Una instancia de `RunWorkout` puede ser utilizada en cualquier lugar donde se espera un `Workout`. Esta es la esencia del **polimorfismo**: poder tratar a distintos objetos como si fueran del mismo tipo base, siempre que respondan a los mismos métodos.

```

1  def total_calories(workouts):
2      return sum(w.get_calories() for w in workouts)
3
4  def total_elevation(run_workouts):
5      return sum(w.get_elev() for w in run_workouts)

```

La primera función suma calorías de cualquier tipo de entrenamiento. La segunda, en cambio, solo funciona con entrenamientos que tengan el método `get_elev` (por ejemplo, `RunWorkout`).

Override: redefiniendo métodos

Las subclases también pueden **sobrescribir** métodos de la clase padre. Por ejemplo, si `RunWorkout` tiene acceso a los puntos GPS de una ruta, puede calcular calorías quemadas con mayor precisión:

```

1  class RunWorkout(Workout):
2      cals_per_km = 100

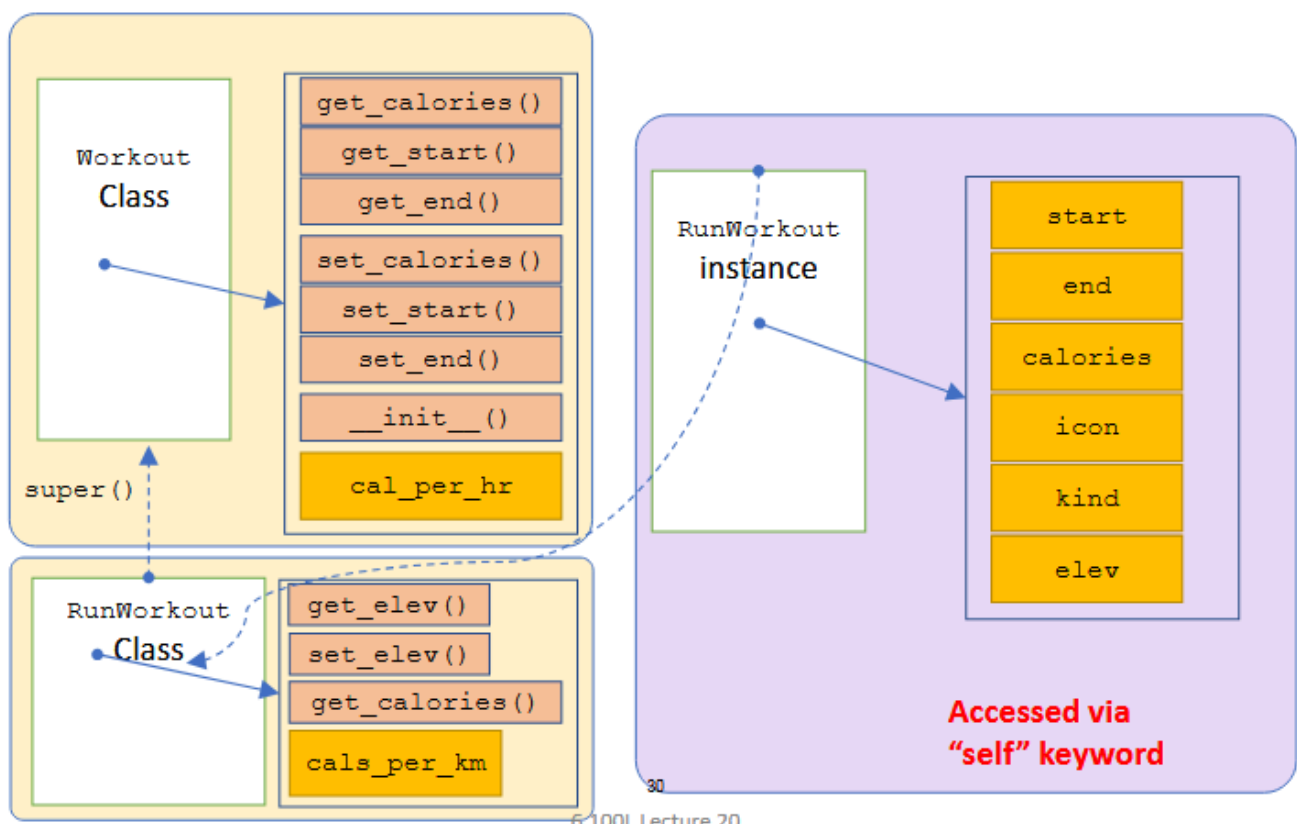
```

```

3
4     def get_calories(self):
5         if self.route_gps_points is not None:
6             dist = 0
7             lastP = self.route_gps_points[0]
8             for p in self.route_gps_points[1:]:
9                 dist += gpsDistance(lastP, p)
10                lastP = p
11            return dist * RunWorkout.cals_per_km
12        else:
13            return super().get_calories()

```

El método `get_calories()` primero intenta usar la ruta para un cálculo más exacto; si no puede, recurre al comportamiento de la clase base. Esto es **herencia dinámica** en acción.



¿Qué método se ejecuta?

Cuando un método es llamado en una instancia, Python busca primero en su clase. Si no lo encuentra, sube por la jerarquía hasta hallarlo. Este comportamiento es vital para entender cómo y cuándo se activan los métodos sobrescritos en las subclases.

Comparando objetos

A veces necesitamos verificar si dos entrenamientos son "iguales". Para ello, definimos el método especial `__eq__` en cada clase:

```
1 class Workout:
2     def __eq__(self, other):
3         return type(self) == type(other) and \
4             self.startDate == other.startDate and \
5             self.endDate == other.endDate and \
6             self.kind == other.kind and \
7             self.get_calories() == other.get_calories()
8
9 class RunWorkout(Workout):
10     def __eq__(self, other):
11         return super().__eq__(other) and self.elev == other.elev
```

El método en `RunWorkout` extiende la comparación base sumando el atributo `elev`.

Ejercicio manual de la lectura

En este problema, vas a implementar dos clases según la especificación que se detalla a continuación: una clase `Container` y una clase `Queue` (una subclase de `Container`).

Nuestra clase `Container` inicializará una lista vacía. Tendrá dos métodos: uno para calcular el tamaño de la lista y otro para agregar un elemento. El segundo método será heredado por la subclase. Ahora queremos crear una subclase para añadir más funcionalidad: la capacidad de eliminar elementos de la lista. Una `Queue` (cola) agregará elementos a la lista de la misma forma, pero se comportará de manera diferente al eliminar un elemento.

Una cola es una estructura de datos **FIFO** (first-in, first-out; primero en entrar, primero en salir). Pensá en la fila para pagar en un comercio. El cliente que lleva más tiempo en la fila es quien pasa al próximo cajero disponible. Al implementar tu clase `Queue`, vas a tener que pensar qué extremo de la lista contiene el elemento que ha estado allí por más tiempo. Ese es el elemento que deberás eliminar y retornar.

```
1 class Container(object):
2     """
3     A container object is a list and can store elements of any type
4     """
5     def __init__(self):
6         """
```

```

7         Initializes an empty list
8         """
9         self.myList = []
10
11     def size(self):
12         """
13         Returns the length of the container list
14         """
15         return len(self.myList)
16
17     def add(self, elem):
18         """
19         Adds the elem to one end of the container list, keeping the end
20         you add to consistent. Does not return anything
21         """
22         self.myList.append(elem)
23
24     class Queue(Container):
25         """
26         A subclass of Container. Has an additional method to remove elements.
27         """
28         def remove(self):
29             """
30             The oldest element in the container list is removed
31             Returns the element removed or None if the stack contains no
32             elements
33             """
34             if len(self.myList) == 0:
35                 return None
36             else:
37                 return self.myList.pop(0)

```